SUMMARY: Strings

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand the concept of indexing and slicing.
- Understand the common strings methods and operations

KEY POINTS AND EXAMPLES:

Understand the concept of indexing and slicing:

Indexing: Indexing in Python strings refers to the process of accessing individual characters within a string by their position, known as the index. In Python, strings are zero-indexed, meaning the index of the first character is 0, the second character is at index 1, and so on. Negative indexing starts from -1, where the last character of the string is at index -1, the second last character is at index -2, and beyond.

	P	Y	T	H	O	N	
Positive Indexing →	0	1	2	3	4	5	
	-6	-5	-4	-3	-2	-1	← Negative Indexing

Each character in the string is assigned a unique index position, including spaces and special characters such as commas and exclamation marks.

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Here's the syntax for accessing characters using indexing in Python strings.

```
string[index]
```

For example,

```
string = "Hello, World!"
# Accessing individual characters
first_char = string[0]
second_char = string[1]  '
last_char = string[-1]
print("First character:", first_char)
print("Second character:", second_char)
print("Last character:", last_char)
```

The output will be:

```
First character: H
Second character: e
Last character: !
```

In these examples:
string[0] accesses the first character of the string.
string[1] accesses the second character of the string.
string[-1] accesses the last character of the string.

```
string = "Hello"
print(string[-6])  # Will raise an IndexError
```

The output will be:

```
IndexError: string index out of range
```

Attempting to access a string using a negative index that is out of bounds will result in an IndexError. In this example, the string "Hello" has a length of 5 characters, so accessing it with an index of -6 (which refers to the 6th character from the end) is out of bounds and raises an error.

Slicing: Slicing allows you to extract substrings from a string by specifying a range of indices. It's important to note that slicing does not include the character at the last index position.

Here's the syntax for accessing substrings using slicing.

```
string[start:end:step]
```

string: The string from which to extract the substring.
start: The index position of the first character of the substring (inclusive).
end: The index position of the character just after the last character of the substring (exclusive).
step: (Optional) The step value, which determines how many characters are included in the substring at each step.

By default, the step parameter is set to 1, meaning that consecutive characters are included in the substring. However, we can specify a different step value to skip characters or reverse the substring.

If the ending index is not specified in slicing, Python assumes that you want to slice from the starting index to the end of the string.

If the starting index is not specified in slicing, Python assumes that you want to slice from the beginning of the string.

For examples,

```
# Define a string
string = "Hello, World!"
# Slicing to extract substrings
substring1 = string[1:5]# Extract characters from index 1 to 5
(not including)
substring2 = string[7:] # Extract characters from index 7 to the
end of the string
```

```
substring3 = string[:5] # Extract characters from the beginning of
the string to index 5 (not including)

# Display the substrings
print(substring1)
print(substring2)
print(substring3)
```

The output will be:

```
ello
World!
Hello
```

```
string = "Hello"
print(string[1:10])
```

The output will be:

```
ello
```

Python does not raise an error when slicing a string with an end index that is out of bounds because it handles such cases gracefully by returning the portion of the string that is within bounds. In this example, the string "Hello" has a length of 5 characters. When slicing from index 1 to index 10, Python simply returns the characters from index 1 to the end of the string, which is "ello".

```
string = "Hello"
print(string[6:10])
```

This code will output an empty string because the start index (6) is out of bounds. In Python, accessing elements beyond the length of a string using slicing does not raise an error; instead, it simply returns an empty string.

Note: Python's slicing convention dictates that the starting index should come before the ending index.

```
string = "Python"
```

```
print(string[-3:0])
```

When using negative indices in slicing, Python counts backward from the end of the string. So, in the expression string[-3:0], the index -3 corresponds to the letter 'h', and the index 0 corresponds to the beginning of the string. In this case, the starting index -3 comes after the ending index 0, which is why it results in an empty string.

```
string = "Hello, World!"
substring = string[1:10:2]  # Includes every second character from
index 1 to 10 (not including)
print(substring)
```

The output will be:

```
el,Wr
```

```
# Define a string
string = "Python Programming is Fun!"
# Negative indexing and slicing
substring1 = string[-4:]  # Extract the last 4 characters
substring2 = string[:-5]  # Extract all characters except the last
5
substring3 = string[::-1]  # Reverse the string

# Display the substrings
print(substring1)
print(substring2)
print(substring3)
```

The output will be:

```
Fun!
Python Programming is
!nuF si gnimmargorP nohtyP
```

In this example, string[::-1] slices the entire string with a step of -1, effectively reversing it. Since neither the start nor the end index is specified, Python uses the entire string by default.

Understand the common strings methods and operations:

Length: Returns the length (number of characters) of the string including any whitespace characters such as spaces.

Syntax: len(string)

```
string = "Hi, there"

# Find the length of the string
length = len(string)

# Print the length
print( length)
```

The output will be:

```
9
```

Case Conversion: In Python, strings can be converted to different case formats, such as uppercase, lowercase, title case, or capitalized case.

1. **Uppercase:** Converts all characters in the string to uppercase.

```
string = "Hi, there"

# Convert to uppercase
uppercase_string = string.upper()

# Print the result
print( uppercase_string)
```

The output will be:

```
HI, THERE
```

2. **Lowercase:** Converts all characters in the string to lowercase.

```
string = "Hi, there"

# Convert to lowercase
```

```
lowercase_string = string.lower()

# Print the result
print(lowercase_string)
```

The output will be:

```
hi, there
```

3. **Title Case:** Converts the first character of each word in the string to uppercase, while converting all other characters to lowercase.

```
string = "Python Programming is Fun!"

# Convert to title case
titlecase_string = string.title()
print(titlecase_string)
```

The output will be:

```
Python Programming Is Fun!
```

4. **Capitalized Case:** Converts the first character of the string to uppercase, while leaving all other characters unchanged.

```
string = "Python Programming is Fun!"
# Convert to capitalized case
capitalized_string = string.capitalize()
print(capitalized_string)
```

The output will be:

```
Python programming is fun!
```

Concatenation: The + operator is used to concatenate strings together. When applied to strings, it joins them together to create a new string.

```
str1 = "Hello"
```

```
str2 = "World"
result = str1 + str2
print(result)
```

The output will be:

```
HelloWorld
```

In Python, we can concatenate a string with an integer/float using the + operator. However, before concatenating, we need to convert the integer/float to a string using the str() function.

```
age = 25
print("Age: " + str(age))
```

The output will be:

```
Age: 25
```

```
name = "John"
age = 22
message = "Hello, " + name + "! You are " + str(age) + " years
old."
print(message)
```

The output will be:

```
Hello, John! You are 22 years old.
```

Repeating/Replicating Strings: The * operator, when applied to a string, repeats the string a certain number of times.

```
str1 = "abc"
result = str1 * 3
print(result)
```

The output will be:

```
abcabcabc
```

Comparing Strings: In string comparison, we aim to identify whether two strings are equivalent to each other and if not, which one is greater.
String comparison in Python takes place character by character. That is, characters in the same positions are compared from both the strings.
If the characters fulfill the given comparison condition, it moves to the characters in the next position. Otherwise, it merely returns False.
Note: Some points to remember when using string comparison operators:

- The comparisons are case-sensitive, hence same letters in different letter cases(upper/lower) will be treated as separate characters.
- If two characters are different, then their Unicode value is compared; the character with the smaller Unicode value is considered to be lower. This is done using the comparison operators

```
string1 = "apple"
string2 = "banana"
# Equal to
print(string1 == string2)
# Not equal to
print(string1 != string2)
# Less than
print(string1 < string2)
# Greater than
print(string1 > string2)
# Less than or equal to
print(string1 <= string2)
# Greater than or equal to
print(string1 >= string2)
```

The output will be:

```
False
True
True
False
True
False
```

For the above example, the Unicode value for the character 'a' is 97, and the Unicode value for the character 'b' is 98. Therefore, 'a' from "apple" has a lower Unicode code point

compared to 'b' from "banana".

If you compare "apple" and "apricot", the first two characters are the same ('a' and 'p'). The comparison then proceeds to the third character, 'p' in "apple" and 'r' in "apricot", where it finds a difference.
This is where Python determines which string is "less than" or "greater than" based on Unicode values.

replace(): Strings are immutable, which means that we cannot change the value of individual characters in a string once it has been created. However, the replace() method can be used to create a new string with some of the characters in the original string replaced with different characters.

```
# Original string
original_string = "Hello, world!"

# Replace "world" with "Python"
new_string = original_string.replace("world", "Python")

# Print the new string
print(new_string)
```

The output will be:

```
Hello, Python!
```

The replace() method in Python allows you to replace not only substrings but also individual characters within a string.

```
# Original string
original_string = "Hello, world!"

# Replace 'o' with 'x'
new_string = original_string.replace('o', 'x')

# Print the new string
print(new_string)
```

The output will be:

```
Hellx, wxrld!
```

split(): It splits the original string into individual words based on the whitespace between them.

```
# Original string
sentence = "This is a sentence."

# Split the string into words based on whitespace
words = sentence.split()

# Print the words
print(words)
```

The output will be:

```
['This', 'is', 'a', 'sentence.']
```

```
# Original string
a = "John,Doe,30"

# Split the string into fields based on comma
fields = a.split(',')

# Print the list of fields
print(fields)
```

The output will be:

```
['John', 'Doe', '30']
```

```
string = "HelloWorld"
print(string.split('-'))
```

The output will be:

```
['HelloWorld']
```

This code will output ['HelloWorld'] because the split() method attempts to split the string

using the specified delimiter '-', but since the delimiter '-' is not found in the string, the entire string is returned as a single element in the result.

Formatting: We cannot combine strings and numbers directly like this as it will result in an error.

```
age = 36
txt = "My name is Alice, I am " + age
print(txt)
```

But we can combine strings and numbers by using f-strings or the format() method.

The format() method and f-strings in Python are used to format strings by inserting values into placeholders within the string. These placeholders are typically represented by curly braces `{}`.

Using format() method:

```
age = 36
name = "Alice"
txt = "My name is {}, I am {} years old.".format(name, age)
print(txt)
```

Using f-strings:

```
age = 36
name = "Alice"
txt = f"My name is {name}, I am {age} years old."
print(txt)
```

The output will be:

```
My name is Alice, I am 36 years old.
```

strip(): The strip() method by default removes any whitespace from the beginning or the end. Additionally, it can be used to remove specific characters.

```
text = "   Hello, World!   "
stripped_text = text.strip()
print(stripped_text)
```

The output will be:

```
Hello, World!
```

```
string = "**Hello**"
result = string.strip('*')
print(result)
```

The output will be:

```
Hello
```

index(): The index() method in Python is used to find the index of the first occurrence of a substring within a string. If the substring is not found, it raises a ValueError.

```
string = "Hello, World!"
index = string.index("World")
print(index)
```

The output will be:

```
7
```

```
string = "Hello, World!"
index = string.index("Python")
```

The output will be:

```
ValueError: substring not found
```

find(): The find() method finds the first occurrence of the specified value within the string. If the value is not found, the find() method returns -1. The find() method also supports optional parameters for specifying the start and end indices for the search.

```
sentence = "This is a sample sentence."
index = sentence.find("sample")
```

```
print(index)
```

The output will be:

```
10
```

In this example, the find() method searches for the substring "sample" within the string sentence. Since "sample" starts at index 10 in the string, the method returns 10 as the output.

```
sentence = "This is a sample sentence."
index = sentence.find("is", 3, 10)
print(index)
```

The output will be:

```
5
```

In this example, the search for "is" is constrained to the indices between 3 and 10 in the string sentence. Since "is" is found at index 5 within this range, the method returns 5 as the output.

The difference between find() and index():

The primary difference between the find() and index() methods in Python lies in how they handle situations where the substring is not found within the string

Method	**Behavior when substring is not found**	**Error raised**
find()	Returns -1	No
index()	Raises ValueError	Yes

startswith(): The startswith() method in Python is used to check if a string starts with a specified prefix. It returns True if the string starts with the specified prefix; otherwise, it returns False.

Syntax:

```
string.startswith(prefix, start, end)
```

- prefix: The substring to check if the string starts with.
- start (optional): The starting index within the string where the search starts. Default is

0.

- end (optional): The ending index within the string where the search ends. Default is the end of the string.

```
string = "Hello, world!"
print(string.startswith("Hello"))  # Output: True
print(string.startswith("hello"))  # Output: False(case-sensitive)
print(string.startswith("world", 7))  # Output: True (start
searching from index 7)
```

count(): The count() method in Python is used to count the number of occurrences of a substring within a string. It returns the count of how many times the specified substring appears in the string. The count() method also supports optional parameters for specifying the start and end indices for the count. If the substring is not found in the string, the method returns 0.

```
sentence = "This is a sample sentence."
count = sentence.count("is")
print(count)
```

The output will be:

```
2
```

isalpha(): The isalpha() method in Python is used to check if all the characters in a string are alphabetic. It returns True if all characters in the string are alphabetic (consist only of letters), otherwise it returns False.

```
text1 = "Hello"
text2 = "Hello123"
print(text1.isalpha())
print(text2.isalpha())
```

The output will be:

```
True
False
```

isalnum(): The isalnum() method is used to check if all the characters in a string are alphanumeric, meaning they consist of either letters (alphabetic characters) or digits (numeric

characters). It returns True if all characters in the string are alphanumeric, otherwise it returns False.

```
text1 = "Hello123"
text2 = "Hello 123"
print(text1.isalnum())
print(text2.isalnum())
```

The output will be:

```
True
False
```

In this example, text1 consists only of alphanumeric characters, so text1.isalnum() returns True. However, text2 contains a space character along with alphanumeric characters, so text2.isalnum() returns False.

isupper(): The isupper() method in Python is used to check if all the characters in a string are uppercase letters. It returns True if all characters in the string are uppercase, otherwise it returns False.

```
text1 = "HELLO"
text2 = "Hello"
print(text1.isupper())
print(text2.isupper())
```

The output will be:

```
True
False
```

islower(): The islower() method in Python is used to check if all the characters in a string are lowercase letters. It returns True if all characters in the string are lowercase, otherwise it returns False.

```
text1 = "hello"
text2 = "Hello"
print(text1.islower())
print(text2.islower())
```

The output will be:

```
True
False
```

isnumeric(): The isnumeric() method in Python is used to check if all the characters in a string are numeric. It returns True if all characters in the string are numeric (digits), otherwise it returns False.

```
text1 = "12345"
text2 = "123abc"
print(text1.isnumeric())
print(text2.isnumeric())
```

The output will be:

```
True
False
```

Note: spaces are not considered numeric by the isnumeric() method.

isdigit(): The isdigit() method in Python is used to check whether all the characters in a string are digits. It returns True if all characters in the string are digits and there is at least one character, otherwise it returns False.

```
string1 = "12345"
string2 = "abc123"
string3 = "123 456"
print(string1.isdigit())
print(string2.isdigit())
print(string3.isdigit())
```

The output will be:

```
True
False
False
```

string3 contains a space, so isdigit() returns False as well.

swapcase(): The swapcase() method in Python is used to swap the case of alphabetic characters in a string. It converts all lowercase characters to uppercase and all uppercase characters to lowercase. Characters that are not alphabetic remain unchanged.

```
text = "Hello World! 123"
swapped_text = text.swapcase()
print(swapped_text)
```

The output will be:

```
hELLO wORLD! 123
```